人体关键点检测

人体关键点检测
- 例程实验效果
- 代码讲解
  - 代码结构
    - 代码解析
    - 流程图
- 人体关键点检测算法简述
  - 常见的应用场景
  - 算法简述
    - 网络结构

例程实验效果

本节我们来学习一下使用K230实现人体关键点检测的功能。

例程代码在【 源码汇总 / 08.Body / 02.person_keypoint_detect.py】

连接IDE后运行本节例程代码，用K230对准一张有多个人体存在的画面，可以看到屏幕中标记出了人体关键点的位置。对于多个人体的场景，也能较为精准的识别出来



代码讲解

代码结构

主程序流程：

1. 初始化阶段：
    - 初始化Pipeline
    - 初始化PersonKeyPointApp类
    - 配置预处理参数
2. 主循环：
    - 获取图像帧
    - 执行模型推理
    - 绘制检测结果
    - 显示图像
    - 执行垃圾回收
3. 异常处理：
    - 检测异常情况
    - 正常则继续循环
    - 异常则退出程序

代码解析

完整代码请参考【源码汇总 / 08.Body / 02.person_keypoint_detect.py】文件

```
if __name__ == "__main__":
    # 显示模式，默认"hdmi"，可以选择"hdmi"和"lcd"
    # Display mode, default "hdmi", can choose "hdmi" or "lcd"
    display_mode = "lcd"
    rgb888p_size = [640, 360]  # RGB图像尺寸 (RGB image size)

    if display_mode == "hdmi":
        display_size = [640, 360]  # HDMI显示尺寸 (HDMI display size)
    else:
        display_size = [640, 480]  # LCD显示尺寸 (LCD display size)

    # 模型路径 (Model path)
    kmodel_path = "/sdcard/kmodel/yolov8n-pose.kmodel"
    # 其它参数设置 (Other parameter settings)
    confidence_threshold = 0.2  # 置信度阈值 (Confidence threshold)
    nms_threshold = 0.5  # 非极大值抑制阈值 (NMS threshold)

    # 初始化PipeLine (Initialize PipeLine)
    pl = PipeLine(rgb888p_size=rgb888p_size, display_size=display_size,
display_mode=display_mode)
    pl.create()

    # 初始化自定义人体关键点检测实例 (Initialize custom person keypoint detection
instance)
    person_kp = PersonKeyPointApp(
        kmodel_path,
        model_input_size=[320, 320],
        confidence_threshold=confidence_threshold,
        nms_threshold=nms_threshold,
        rgb888p_size=rgb888p_size,
        display_size=display_size,
        debug_mode=0
```

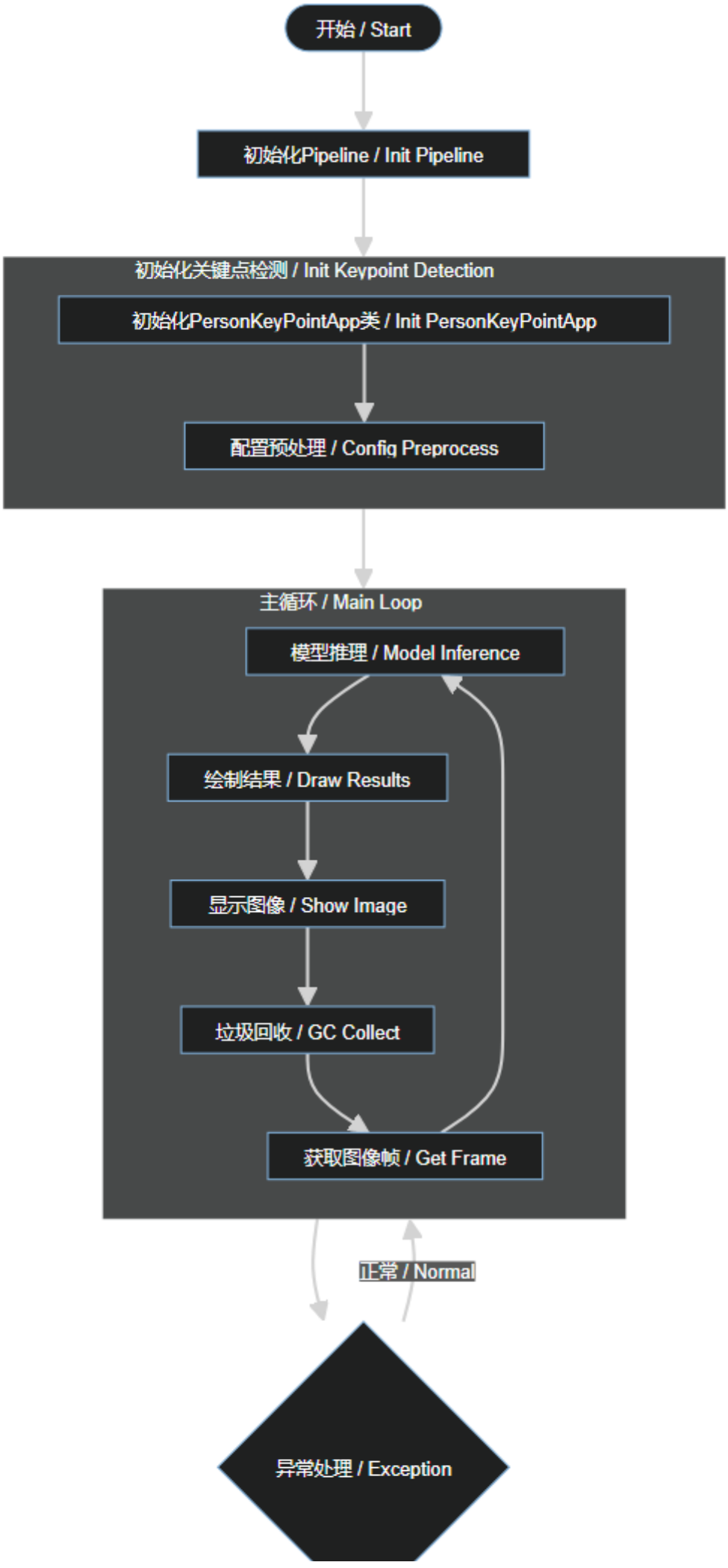
```
    )

    person_kp.config_preprocess()  # 配置预处理 (Configure preprocessing)

    while True:
        with ScopedTiming("total", 1):
            # 获取当前帧数据 (Get current frame data)
            img = pl.get_frame()
            # 推理当前帧 (Infer current frame)
            res = person_kp.run(img)
            # 绘制结果到PipeLine的osd图像 (Draw results to PipeLine's osd image)
            person_kp.draw_result(pl, res)
            # 显示当前的绘制结果 (Show current drawing results)
            pl.show_image()
            gc.collect()  # 垃圾回收 (Garbage collection)

    # 释放资源 (Release resources)
    person_kp.deinit()
    pl.destroy()
```

流程图


开始 / Start
初始化Pipeline / Init Pipeline
初始化关键点检测 / Init Keypoint Detection
初始化PersonKeyPointApp类 / Init PersonKeyPointApp
配置预处理 / Config Preprocess
主循环 / Main Loop
模型推理 / Model Inference
绘制结果 / Draw Results
显示图像 / Show Image
垃圾回收 / GC Collect
获取图像帧 / Get Frame
正常 / Normal
异常处理 / Exception



人体关键点检测算法简述

常见的应用场景

智能健身和运动分析

- 姿势矫正和动作指导
- 运动表现分析和评估
- 远程健身教练系统

医疗康复

- 患者运动功能评估
- 康复训练进度跟踪
- 远程医疗指导

安防监控

- 异常行为识别
- 跌倒检测和预警
- 人群密度和流动分析

人机交互

- 手势识别控制
- 虚拟现实/增强现实交互
- 动作捕捉和3D建模

体育竞技

- 运动员技术动作分析
- 比赛战术研究
- 训练辅助系统

工业生产

- 工作环境安全监测
- 工人操作规范性检查
- 人机协作控制

算法简述

网络结构

人体关键点检测通常采用以下网络架构：

主干网络(Backbone)

- 使用深度卷积神经网络 (如ResNet、HRNet等) 提取图像特征
- 通过多尺度特征提取捕获不同尺度的人体特征信息

颈部网络(Neck)

- 特征金字塔结构(FPN)整合不同层级的特征
- 上采样和下采样结合，增强特征表达能力

预测头(Head)

- 生成热力图(Heatmap)预测每个关键点的位置概率
- 输出通道数等于关键点数量(如COCO数据集17个关键点)